FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação

6º Semestre — Turma 6º CCOMP

SWING SENSE

DOCUMENTAÇÃO FORMAL DAS REGRAS DE FUNCIONAMENTO DO SISTEMA

Entrega 2 — Teoria da Computação e Linguagens Formais

Grupo 1

Antônio Petri de Moraes Soares de Moura e Oliveira

Gabriel Higobassi Paschoal

Lucas Silva Maciel

Thiago Akira Higa Mitami

São Paulo

2026

1 INTRODUÇÃO

Na Entrega 1, o módulo de detecção de tacadas do Swing Sense foi modelado como um autômato finito com quatro estados (PARADO, PREPARACAO, IMPACTO e RECUPERACAO), alfabeto Σ = {g, a, t, e} e linguagem L = (g(ae|t))*. Desde então o firmware do ESP32-C3 evoluiu: a versão atual (Passo 3) retirou o saque da classificação, passou a distinguir forehand e backhand pela aceleração no eixo X antes do impacto, ganhou um segundo tempo-limite na fase de recuperação, passou a reportar a força do golpe apenas quando ele termina e envia cada tacada ao aplicativo por Bluetooth Low Energy (BLE), em formato JSON.

Este documento corresponde à Entrega 2 da disciplina e documenta formalmente as regras de funcionamento do sistema: a máquina de estados revisada, a descrição de todos os eventos, as regras que controlam o comportamento, as pré e pós-condições das operações principais, a justificativa do modelo computacional adotado e a análise de decidibilidade das decisões tomadas pelo sistema. Todas as constantes, estados e condições citados foram extraídos do código-fonte atual do firmware, de modo que o modelo pode ser conferido linha a linha contra a implementação.

2 ATUALIZAÇÃO DA MÁQUINA DE ESTADOS

A estrutura central do autômato foi mantida, mas a evolução do firmware provocou alterações no alfabeto, nas transições de saída da recuperação, nas ações executadas em cada transição e no escopo do modelo, que agora inclui a inicialização do dispositivo e o enlace BLE. O Quadro 1 resume as diferenças.

2.1 Alterações em relação à Entrega 1

Quadro 1 — Alterações do sistema entre a Entrega 1 e a Entrega 2

Aspecto	Entrega 1 / versões anteriores	Entrega 2 (firmware atual)
Tipos de golpe	SAQUE, FOREHAND e BACKHAND.	FOREHAND e BACKHAND. O saque foi retirado do MVP depois de falhar nos testes de campo.

Aspecto	Entrega 1 / versões anteriores	Entrega 2 (firmware atual)
Regra de classificação	Comparação de eixos (aceleração e giro em Z) no instante do impacto.	Média de accX das 5 amostras anteriores ao impacto (buffer circular de 16 posições): BACKHAND se maior que 14,0 m/s², senão FOREHAND. Golpe marcado como "incerto" se a média estiver a menos de 4,0 m/s² do limiar.
Saída de RECUPERACAO	Apenas por estabilização da aceleração (evento e).	Por estabilização (e) ou por tempo-limite de 2000 ms (novo evento r).
Momento do relatório	No impacto, junto com a classificação.	Classificação no impacto; força, pico de giro e velocidade são reportados ao sair de RECUPERACAO, quando o pico real (≈100 ms após o cruzamento do limiar) já foi capturado.
Faixas do sensor	Faixas menores: os registros de campo (planilha "Dadosbatidas") saturam em 8,731 rad/s, isto é, ±500 °/s.	±16 g no acelerômetro, ±2000 °/s no giroscópio e filtro passa-baixa de 21 Hz.
Inicialização	Não modelada.	Estados N (inicialização) e X (erro de sensor) explicitados.
Comunicação	Somente Monitor Serial.	Serial e notificação BLE em JSON (contrato v1), modelada como componente paralelo com estados ANUNCIANDO e CONECTADO.
Identificação da tacada	Inexistente.	Cada tacada recebe boot (sorteado a cada vez que o ESP32 liga) e seq (1, 2, 3...).

Fonte: elaborado pelos autores (2026).

2.2 Definição formal

O comportamento de detecção passa a ser descrito pela quíntupla M = (Q, Σ, δ, q_0, F), acrescida de uma função de saída λ, o que caracteriza uma máquina de Mealy (autômato finito com saída):

$$Q = \{ N, X, P, R, I, C \} \qquad \Sigma = \{ k, f, g, a, t, e, r \}$$

$$q_0 = N \qquad F = \{ P \}$$

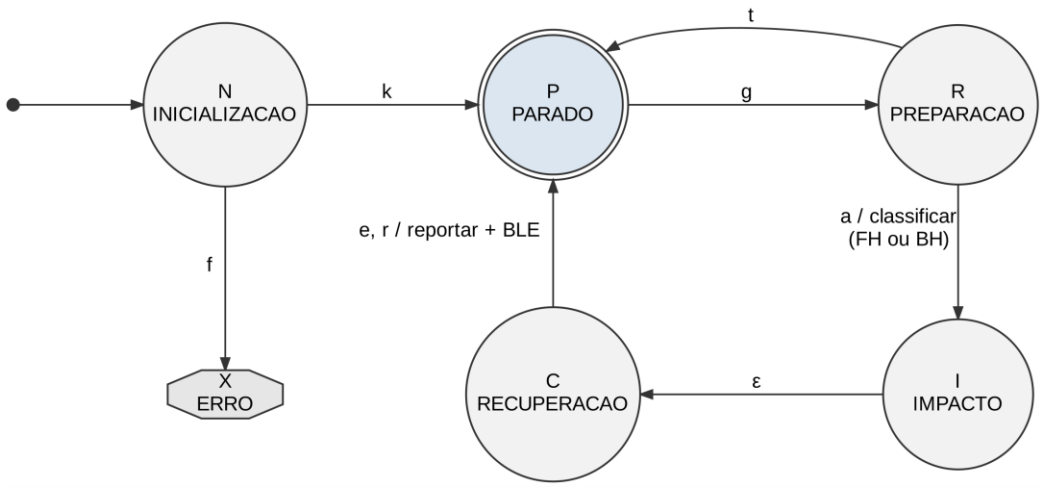
Os estados N (INICIALIZACAO) e X (ERRO) são novos; P, R, I e C mantêm o significado da Entrega 1. P continua sendo o único estado de aceitação, pois representa o sistema pronto para uma nova tacada após um ciclo completo. X é um estado de armadilha: não possui transições de saída, o que corresponde ao laço `while (1)` executado quando o MPU6050 não é encontrado. A transição de IMPACTO para RECUPERACAO continua espontânea (ε), pois ocorre na iteração seguinte do laço principal sem consumir nenhum evento.

Seguindo a convenção da Entrega 1, a ausência de evento em uma iteração do laço não consome símbolo: o sistema simplesmente permanece no estado atual. Assim, a cadeia de entrada é formada apenas pelos eventos que efetivamente provocam mudança de estado. A função de saída λ associa ações a três transições: λ(R, a) = classificar o golpe; λ(C, e) = λ(C, r) = reportar a tacada (Serial e BLE).

2.3 Diagrama de estados revisado

A Figura 1 apresenta o diagrama revisado. Os rótulos no formato "evento / ação" indicam a saída produzida na transição. Em relação à Entrega 1, destacam-se a transição de RECUPERACAO para PARADO, agora rotulada "e, r" (o evento r é novo), e os estados de inicialização e erro à esquerda.

Figura 1 — Diagrama de estados revisado do sistema de detecção de tacada



Fonte: elaborado pelos autores (2026).

2.4 Tabela de transições

Quadro 2 — Função de transição δ e função de saída λ

Estado atual	Evento	Próximo estado	Ação executada (saída)
N	k	P	Configura o MPU6050 (±16 g, ±2000 °/s, 21 Hz), sorteia idBoot e inicia o anúncio BLE como SwingSense-XXXX.
N	f	X	Imprime "ERRO: nao encontrou o MPU6050" e entra em laço infinito.
P	g	R	Registra marcaTempo e zera os picos de aceleração e de giro.
R	a	I	classificarNoImpacto(): define FOREHAND ou BACKHAND, incrementa o contador do tipo e registra tempoImpacto.

Estado atual	Evento	Próximo estado	Ação executada (saída)
R	t	P	Nenhuma: a tentativa é descartada sem classificação nem envio.
I	ε	C	Atualiza os picos, registra marcaTempo e zera tempoEstavelDesde.
C	e	P	reportarTacada(): incrementa seq, monta o JSON, imprime no Serial e envia por BLE.
C	r	P	reportarTacada(), encerrando a tacada por tempo esgotado.

Fonte: elaborado pelos autores (2026).

2.5 Linguagem, expressão regular e gramática atualizadas

Com o novo evento r, cada ciclo de detecção bem-sucedido passa a terminar por "e" ou por "r". A linguagem do módulo de detecção, a partir de PARADO, e a linguagem do sistema completo, a partir da inicialização, são:

$$L_D = (g (a (e \mid r) \mid t))^*$$

$$L = k (g (a (e \mid r) \mid t))^*$$

Exemplos de sequências válidas: `k` (sistema ligado, nenhuma tacada); `k gae` (uma tacada completa); `k gar gt gae` (uma tacada encerrada por tempo-limite, uma tentativa cancelada e uma tacada normal). Exemplos de sequências inválidas: `f` (termina em X, que não é de aceitação); `gae` (sem inicialização); `k ge` (após "g" só são possíveis "a" ou "t"); `k ga` (termina em RECUPERACAO, ciclo incompleto).

A gramática regular linear à direita G = (V, Σ, P, S_0), com V = {S_0, S, A, B}, gera L. Cada não-terminal corresponde a um estado: S_0 a INICIALIZACAO, S a PARADO, A a PREPARACAO e B a IMPACTO/RECUPERACAO (combinados, como na Entrega 1, porque a transição entre eles é ε). O estado X não possui produções, pois nenhuma cadeia aceita passa por ele.

$$S_0 \rightarrow k\ S$$

$$S \rightarrow g\ A \mid \varepsilon$$

$$A \rightarrow a\ B \mid t\ S$$

$$B \rightarrow e\ S \mid r\ S$$

2.6 Componente paralelo: enlace BLE

A conexão com o celular evolui de forma independente da detecção: uma tacada pode ocorrer com ou sem celular conectado, e conectar ou desconectar não

altera a fase do gesto. Por isso o enlace foi modelado como um segundo autômato, M_B = ({A, L}, {c, d}, δ_B, A), executado em paralelo (Figura 2). O evento c ocorre quando um cliente BLE se conecta (variável `bleConectado` passa a verdadeira) e d quando ele se desconecta, retornando o dispositivo ao anúncio.

O sistema completo é a composição paralela M × M_B, com 6 × 2 = 12 configurações possíveis, ainda um número finito. O único ponto de interação entre os dois autômatos é a saída de reportarTacada(): no estado L a tacada é enviada como notificação BLE; no estado A ela aparece apenas no Monitor Serial ("modo ao vivo").

Figura 2 — Autômato do enlace BLE, executado em paralelo à detecção



Fonte: elaborado pelos autores (2026).

3 DESCRIÇÃO DOS EVENTOS

Os sinais do MPU6050 são contínuos (números reais lidos a cada ≈10 ms). O firmware os converte em eventos discretos comparando-os com limiares fixos e com o tempo decorrido; é essa discretização que permite descrever o sistema por um alfabeto finito. O Quadro 3 relaciona todos os eventos que provocam mudança de estado.

Quadro 3 — Eventos do sistema

Símbolo	Evento	Origem	Condição no código	Estado onde atua
k	Sensor inicializado	Inicialização	`mpu.begin()` retorna verdadeiro; faixas configuradas; `bleIniciar()` executado.	N
f	Falha do sensor	Inicialização	`mpu.begin()` retorna falso (MPU6050 ausente ou mal conectado).	N
g	Rotação detectada	Giroscópio	giroMag > LIMIAR_GIRO_PREPARACAO (1,5 rad/s).	P
a	Pico de aceleração (impacto)	Acelerômetro	accelMag > LIMIAR_ACEL_IMPACTO (20,0 m/s²).	R

Símbolo	Evento	Origem	Condição no código	Estado onde atua
t	Tempo-limite da preparação	Temporizador	agora − marcaTempo > TIMEOUT_PREPARACAO (1000 ms), sem impacto.	R
ε	Avanço espontâneo	Interno	Próxima iteração do laço após entrar em IMPACTO.	I
e	Aceleração estabilizada	Acelerômetro e temporizador	accelMag < LIMIAR_ACEL_ESTAVEL (13,5 m/s²) por mais de TEMPO_ESTAVEL_RECUPERACAO (150 ms) contínuos; qualquer amostra acima reinicia a contagem.	C
r	Tempo-limite da recuperação	Temporizador	agora − marcaTempo > TIMEOUT_RECUPERACAO_SEG (2000 ms).	C
c	Celular conectado	BLE	Um cliente BLE conecta ao serviço do dispositivo.	A
d	Celular desconectado	BLE	O cliente BLE encerra a conexão.	L

Fonte: elaborado pelos autores (2026).

Além desses, o sistema executa ações periódicas que não alteram o estado e por isso não pertencem ao alfabeto: a leitura do sensor e o armazenamento de accX no buffer circular, feitos em toda iteração (≈100 Hz) e em qualquer estado; a atualização dos picos de aceleração e de giro durante PREPARACAO, IMPACTO e RECUPERACAO; e a mensagem de sinal de vida "[vivo]" impressa no Serial a cada 5 s, com o estado atual e a situação do BLE.

4 REGRAS DE FUNCIONAMENTO

O Quadro 4 reúne as regras que controlam o comportamento do sistema. Cada regra decorre diretamente da função de transição ou de uma condição explícita no código.

Quadro 4 — Regras de funcionamento do Swing Sense

ID	Regra
RN01	Se o MPU6050 não for detectado na inicialização, o sistema entra em ERRO e nenhuma tacada pode ser detectada, classificada ou enviada; apenas um reinício do dispositivo sai desse estado.
RN02	Uma tacada só pode ser detectada a partir de PARADO e depois de uma PREPARACAO: um pico de aceleração sem rotação prévia (por exemplo, a raquete batendo na mesa) não é reconhecido como impacto.

ID	Regra
RN03	Se não houver impacto em até 1000 ms após o início da preparação, a tentativa é descartada: nada é classificado, contado ou enviado.
RN04	Se, na mesma amostra, a aceleração ultrapassar o limiar de impacto e o tempo-limite da preparação tiver expirado, prevalece o impacto, pois essa condição é testada primeiro.
RN05	Cada tacada é classificada uma única vez, no instante do impacto, e somente como FOREHAND ou BACKHAND: BACKHAND se a média de accX das 5 últimas amostras for maior que 14,0 m/s²; caso contrário, FOREHAND.
RN06	A tacada é marcada como incerta quando a média de accX fica a menos de 4,0 m/s² do limiar (entre 10,0 e 18,0 m/s²). A marcação não altera o tipo atribuído; apenas informa ao aplicativo que a classificação é pouco confiável.
RN07	Força (pico de aceleração), pico de giro e velocidade estimada só são reportados ao final da RECUPERACAO, para incluir o pico real do golpe. A velocidade é estimada como pico de giro × 0,6 m × 3,6 (km/h).
RN08	Nenhuma nova tacada começa antes de o sistema voltar a PARADO: rotações durante IMPACTO ou RECUPERACAO não iniciam nova preparação.
RN09	A RECUPERACAO dura no máximo 2000 ms; esgotado o prazo, a tacada é reportada mesmo sem estabilização. Com isso, todo ciclo de detecção termina em tempo limitado (no máximo ≈1 s de preparação e ≈2 s de recuperação).
RN10	Cada tacada reportada recebe o número de sequência seq, incrementado de 1, e o identificador boot sorteado ao ligar. O par (boot, seq) identifica a tacada de forma única e permite ao aplicativo detectar duplicatas e lacunas.
RN11	Se não houver celular conectado, a tacada aparece apenas no Serial e não é guardada para envio posterior. Como seq é incrementado mesmo assim, o aplicativo percebe uma lacuna na sequência.
RN12	Toda mensagem BLE segue o contrato v1: campos fixos, na ordem v, boot, seq, t, tipo, incerto, bola, acel, giro, vel e accx; tipo ∈ {"FH", "BH"}; bola é sempre null nesta versão.
RN13	A leitura do sensor e o buffer de accX são atualizados em todos os estados, para que a janela pré-impacto esteja disponível no instante da classificação.

Fonte: elaborado pelos autores (2026).

5 PRÉ E PÓS-CONDIÇÕES

O Quadro 5 define, para as principais operações do sistema, o que deve ser verdadeiro antes da operação (pré-condições) e o que passa a ser verdadeiro depois dela (pós-condições). Cada operação corresponde a uma ou mais transições do autômato.

Quadro 5 — Pré e pós-condições das operações principais

Operação (transição)	Pré-condições	Pós-condições
O1. Inicializar o sistema (N → P ou N → X)	Dispositivo alimentado; estado = N.	Se o sensor respondeu: faixas configuradas, idBoot sorteado, seq = 0, BLE anunciando e estado = P. Caso contrário: estado = X e nenhum evento posterior é processado.
O2. Iniciar preparação (P → R)	Estado = P; giroMag > 1,5 rad/s.	Estado = R; marcaTempo = instante atual; picoAccelMag = picoGiroMag = 0.
O3. Detectar impacto e classificar (R → I)	Estado = R; accelMag > 20,0 m/s²; buffer com pelo menos uma amostra de accX.	tipoAtual ∈ {FOREHAND, BACKHAND}; contador do tipo incrementado em 1; tempoImpacto registrado; estado = I.
O4. Cancelar preparação (R → P)	Estado = R; mais de 1000 ms desde a entrada em R; nenhuma amostra acima de 20,0 m/s² nesse intervalo.	Estado = P; contadores e seq inalterados; nenhuma mensagem enviada.
O5. Avançar para recuperação (I → C)	Estado = I.	Estado = C; marcaTempo = instante atual; tempoEstavelDesde = 0; picos atualizados com a amostra atual.
O6. Encerrar e reportar tacada (C → P)	Estado = C; aceleração abaixo de 13,5 m/s² por mais de 150 ms (e) ou mais de 2000 ms em C (r).	seq incrementado em 1; JSON montado com os picos completos; tacada impressa no Serial; estado = P.
O7. Enviar tacada ao aplicativo	JSON gerado por O6; enlace em L (celular conectado, com notificações ativadas).	O aplicativo recebe uma notificação com a mensagem; se o enlace estiver em A, nada é enviado e a tacada fica apenas no Serial.
O8. Conectar / desconectar (A ⇄ L)	c: estado do enlace = A e celular dentro do alcance. d: estado do enlace = L.	c: bleConectado = verdadeiro, estado = L. d: bleConectado = falso, estado = A. Em ambos os casos o estado de detecção não se altera.

Fonte: elaborado pelos autores (2026).

6 MODELO COMPUTACIONAL

Entre os modelos estudados na disciplina (autômato finito, autômato com pilha e máquina de Turing), o que melhor representa o Swing Sense é o **autômato finito determinístico**, na forma de máquina de Mealy (com saída), composto em paralelo com o autômato do enlace BLE. A escolha se apoia em três argumentos.

O controle depende apenas do estado atual e do evento atual. Em cada iteração, o firmware decide a próxima ação olhando somente para `estadoAtual` e para a amostra corrente convertida em evento. A memória necessária para essa decisão é fixa: seis estados de detecção e dois de enlace. As demais variáveis do programa não alteram esse quadro. Os temporizadores (marcaTempo e

tempoEstavelDesde) servem apenas para gerar os eventos t, e e r, e como seus prazos são fixos (1000, 150 e 2000 ms) poderiam ser substituídos por uma quantidade finita de estados intermediários. O buffer de accX tem tamanho fixo de 16 posições. Já os contadores (contForehand, contBackhand, seq) e os picos são apenas escritos e enviados; nenhum deles é consultado para escolher uma transição, por isso pertencem à função de saída, e não ao controle.

Um autômato com pilha seria superdimensionado. A pilha é necessária quando a linguagem exige casar quantidades sem limite, como em $\{a^n b^n\}$ ou na verificação de parênteses aninhados. O Swing Sense nunca precisa disso: cada ciclo de tacada é independente e se fecha antes do próximo (RN08), e nenhuma regra depende de quantos ciclos ocorreram antes. A linguagem L = k(g(a(e|r)|t))* é regular, como demonstram a expressão regular e a gramática linear à direita da Seção 2.5, e toda linguagem regular é reconhecida por um autômato finito.

Uma máquina de Turing seria desnecessária e indesejável. A máquina de Turing modela computações que exigem memória ilimitada de leitura e escrita. Além de não haver essa necessidade, usá-la como modelo traria propriedades indecidíveis, como o problema da parada, e não ofereceria garantia de tempo de resposta. Para um sistema embarcado que precisa reagir a cada 10 ms, é essencial que cada reação termine em um número limitado de passos, propriedade garantida pelo autômato finito. O ESP32-C3, como computador, tem poder para simular programas muito mais complexos, mas o comportamento modelado não usa esse poder.

7 ANÁLISE DE DECIDIBILIDADE

Um problema é decidível quando existe um procedimento que sempre termina com a resposta correta "sim" ou "não". Todas as decisões tomadas pelo Swing Sense são desse tipo e, além disso, são determinísticas: para a mesma entrada, produzem sempre a mesma resposta. O Quadro 6 lista as principais decisões.

7.1 Decisões realizadas pelo sistema

Quadro 6 — Decisões do sistema e justificativa de decidibilidade

Decisão	Como é tomada	Por que é decidível e determinística
D1. O sensor iniciou?	Valor booleano retornado por `mpu.begin()`.	A chamada retorna sim ou não; não há terceira possibilidade.

Decisão	Como é tomada	Por que é decidível e determinística
D2. Começou um movimento?	giroMag > 1,5 rad/s.	Comparação de um número com uma constante: uma operação, resposta única.
D3. Houve impacto ou a preparação expirou?	Testa accelMag > 20,0 antes de Δt > 1000 ms.	Ordem fixa dos testes: no máximo uma transição é tomada por amostra (RN04).
D4. É forehand ou backhand?	Média das 5 últimas amostras de accX comparada a 14,0 m/s².	Número fixo de operações (5 somas, 1 divisão, 1 comparação); o resultado é sempre um dos dois tipos.
D5. A classificação é incerta?	|média − 14,0| < 4,0.	Uma subtração, um valor absoluto e uma comparação.
D6. O golpe terminou?	Estável por mais de 150 ms ou mais de 2000 ms em C.	Relógio monotônico e prazo máximo: a resposta "sim" sempre chega em até ≈2 s (RN09).
D7. Enviar por BLE?	Valor da variável bleConectado.	Leitura de um booleano.
D8. A mensagem recebida pelo aplicativo é válida?	Casamento com a expressão regular do contrato (Seção 7.3).	Pertinência em linguagem regular é decidível em tempo linear no tamanho da mensagem.

Fonte: elaborado pelos autores (2026).

7.2 Por que as decisões são determinísticas

Quatro propriedades garantem o determinismo. Primeiro, δ é uma função: para cada par (estado, evento) existe no máximo um próximo estado (Quadro 2). Segundo, cada amostra gera no máximo um evento relevante para o estado atual, porque as condições são testadas em ordem fixa. Terceiro, nenhuma decisão contém laço que dependa da entrada: cada iteração do laço principal executa um número limitado de operações e termina. Quarto, todo ciclo de detecção tem duração máxima, pois os tempos-limite t e r forçam a volta a PARADO.

Do ponto de vista formal, verificar se uma sequência de eventos pertence a L é o problema da pertinência para linguagens regulares, que é decidível: basta simular o autômato sobre a cadeia, o que termina após exatamente |w| passos. Em contraste, perguntas sobre programas arbitrários, como saber se um programa qualquer para, são indecidíveis em geral. É justamente por restringir suas decisões a comparações com limiares e a um número finito de estados que o Swing Sense permanece no terreno decidível. O laço principal é infinito por projeto, já que se trata de um sistema reativo; o que importa é que cada reação termine, e isso está garantido.

7.3 Validação da mensagem BLE por expressão regular

JSON genérico, com aninhamento arbitrário de chaves e colchetes, não é uma linguagem regular, pois exige balanceamento (é livre de contexto). O contrato v1 do Swing Sense, porém, é plano e tem campos em ordem fixa, de modo que o conjunto de mensagens válidas é regular. Sendo D = [0-9], N = D+ e F = -?D+\.D, toda mensagem válida casa com a expressão:

```
\{"v":1,"boot":N,"seq":N,"t":N,"tipo":"(FH|BH)",
"incerto":(true|false),"bola":null,"acel":D+\.D,
"giro":D+\.D,"vel":N,"accx":F\}
```

Assim, o aplicativo pode decidir, em tempo linear e sem ambiguidade, se cada notificação recebida é válida, rejeitando mensagens corrompidas ou de outro dispositivo. Também é possível decidir estaticamente que nenhuma mensagem será truncada: com os valores máximos possíveis (boot, seq e t com 10 dígitos; aceleração de até 271,9 m/s², ou seja, 16 g·√3; giro de até 60,5 rad/s; velocidade de até 131 km/h; accX de até −157,0 m/s²), a mensagem mais longa tem 146 bytes, menos que o buffer de 200 bytes do firmware e que a carga útil de 182 bytes de uma notificação BLE com MTU 185.

7.4 Ponto identificado na verificação do código

A comparação do modelo com o código revelou uma divergência. No estado RECUPERACAO, o firmware testa a estabilização e o tempo-limite em sequência, sem `else` nem `break` entre eles. Se as duas condições se tornarem verdadeiras na mesma iteração (a estabilização se completa exatamente quando os 2000 ms expiram), `reportarTacada()` é chamada duas vezes, e a mesma tacada é enviada duas vezes com valores de seq diferentes. No modelo formal isso não ocorre, porque o autômato consome um evento por vez e (C, e) leva a P antes que r possa ser lido. Para que a implementação corresponda exatamente ao modelo, basta transformar o segundo teste em `else if` ou encerrar o caso com `break` logo após o primeiro relatório. Até a correção, o par (boot, seq) não permite ao aplicativo detectar essa duplicata, já que os dois envios têm seq distintos.

8 CONCLUSÃO

A revisão mostrou que as mudanças do firmware entre as duas entregas (fim do saque, nova regra de classificação, tempo-limite na recuperação, relatório ao fim do golpe e envio por BLE) foram absorvidas pelo modelo sem alterar sua classe: o sistema continua sendo descrito por um autômato finito determinístico, agora com saída e composto com o autômato do enlace, e sua linguagem continua regular, L = k(g(a(e|r)|t))*. As regras, pré e pós-condições documentadas tornam explícito o comportamento esperado e as restrições do sistema, e a análise de decidibilidade demonstra que todas as decisões são tomadas em tempo limitado e de forma determinística. A verificação também apontou um caso de relatório duplicado na recuperação, que deve ser corrigido no firmware para que implementação e modelo coincidam.

REFERÊNCIAS

HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. **Introdução à teoria de autômatos, linguagens e computação**. Rio de Janeiro: Elsevier, 2002.

INVENSENSE. **MPU-6000 and MPU-6050 product specification**: revision 3.4. San Jose: InvenSense, 2013.

MENEZES, P. B. **Linguagens formais e autômatos**. 6. ed. Porto Alegre: Bookman, 2011.

SIPSER, M. **Introdução à teoria da computação**. 2. ed. São Paulo: Cengage Learning, 2007.